

Autonomous Content Specialist Crew

This notebook orchestrates an autonomous editorial team that runs entirely within the free tier of Google Gemini, transforming raw developer expertise into high-impact industry content.

- 1. **The Profile Analyst** extracts your core technical strengths, unique engineering perspectives, and authoritative voice.
- 2. **The Market Researcher** analyzes target topics to uncover current industry trends, audience pain points, and content gaps.
- 3. **The Editorial Strategist** cross-references your expertise against market demand, scoring and selecting the highest-impact article angle.
- 4. **The Specialist Copywriter** drafts a polished, highly engaging, and authoritative long-form article or LinkedIn post based on the winning strategy.

```
# Install dependencies
!pip install -q crewai crewai-tools
print('✅ Done')
```

✅ Done

Instructions to Input API Key:

To run this notebook securely without editing a single line of code, add your Gemini API key to your Colab Secrets:

- 1. Click the **Key icon (Secrets)** on the left sidebar of Google Colab.
- 2. Add a new secret named exactly `GEMINI_API_KEY`.
- 3. Paste your free API key from Google AI Studio(<https://aistudio.google.com/api-keys>) into the Value field.
- 4. Toggle the **"Notebook access"** switch to **ON**.

The script below will automatically pull this secret out of your vault, keeping your keys 100% invisible inside the code cells.

```
# Load API key (add GEMINI_API_KEY in Colab secrets)
from google.colab import userdata
import os

# Retrieve the API key from Colab's secrets manager
GEMINI_API_KEY = userdata.get('GEMINI_API_KEY') # Add '#' at beginning if not using this method

# Retrieve the API key from user directly
#os.environ["GEMINI_API_KEY"] = "****Paste YOUR_GEMINI_KEY here****"
# Remove '#' in the line above to execute externally without Google Colab

print('API Key successfully retrieved')
```

API Key successfully retrieved

```
# Imports
from crewai import Agent, Task, Crew, Process, LLM

gemini_llm = LLM(
    model="gemini/gemini-2.5-flash",
    api_key=GEMINI_API_KEY,
    temperature=0.3,
)
print('Imported the required libraries and llm was defined successfully')
```

Imported the required libraries and llm was defined successfully

Your Inputs

Insert your inputs below

```
#Change everything in the quotations (") to your desired preferences to help the AI understand the background

MY_EXPERTISE = """
I am a computer science student with basic knowledge of Python, HTML, CSS,
and programming fundamentals. I am exploring artificial intelligence and
want to understand how modern AI systems are built.
"""

TARGET_TOPICS = [
    "Introduction to large language models",
```

"Building AI applications as a beginner"

}

▼ The Agents

```
expertise_analyst = Agent(
    role="Expertise & Profile Analyst",
    goal="Analyze the author's background and extract their core strengths, unique perspectives, and authoritative voice.",
    backstory="You are an expert personal branding coach. You read an author's background and figure out exactly what they are qualified to talk about.",
    llm=gemini_llm,
    verbose=True
)

topic_researcher = Agent(
    role="Market Topic Researcher",
    goal="Analyze the target topics using your deep internal knowledge of the referred industry to identify current trends, pain points, and valuable insights.",
    backstory="You are a brilliant industry analyst. Instead of searching the web, you draw upon your vast knowledge of the topic, current trends, and audience interests.",
    llm=gemini_llm,
    verbose=True
)

content_fit_scorer = Agent(
    role="Content-Fit Scorer & Strategist",
    goal="Compare the author's expertise against the market research to find the absolute best, highest-scoring angle for an article.",
    backstory="You are a strict Managing Editor. You match what the author knows (Expertise) with what the market wants (Research) and score 1-10 on how well they align.",
    llm=gemini_llm,
    verbose=True
)

content_writer = Agent(
    role="Specialist Content Writer",
    goal="Draft a highly engaging, authoritative article or LinkedIn post based on the winning angle.",
    backstory="You are a world-class copywriter and ghostwriter. You take strategic angles and turn them into compelling, easy-to-read content.",
    llm=gemini_llm,
    verbose=True
)

print('Agents defined successfully')
```

Agents defined successfully

System Guardrail (DO NOT CHANGE)

```
def get_system_guardrail(topics):
    return f'''
    CRITICAL SYSTEM OVERRIDE & COMPLIANCE RULES:
    1. You are strictly forbidden from writing about or researching ANY topic outside of: {topics}.
    2. Do not invent tutorials, do not pivot to local Python tools, and do not hallucinate unrelated concepts.
    3. If your output does not directly address the target topics, you have failed your core directive.
    '''
```

▼ Assigning Tasks to the Agents

```
task_expertise = Task(
    description=f"Analyze this background: {MY_EXPERTISE}. Extract the top 3 unique selling points and areas of authority.",
    expected_output="A bulleted list of the author's core strengths.",
    agent=expertise_analyst
)

task_research = Task(
    description=f"Analyze these target topics: {TARGET_TOPICS}. Identify the biggest trends and what audiences want to read about them right now.",
    expected_output="A market research report detailing current trends.",
    agent=topic_researcher
)

task_score = Task(
    description="Compare the author's strengths against the market research. Generate 3 article angles. Score them out of 100 based on fit, authority, and engagement potential.",
    expected_output="3 angles with scores and a final declaration of the winning angle.",
    agent=content_fit_scorer
)

task_content = Task(
    description="Using the winning angle, write a highly engaging, authoritative 500-word article or long-form LinkedIn post." + get_system_guardrail(TARGET_TOPICS),
    expected_output="A final, polished piece of content in Markdown format.",
    agent=content_writer
)
```

```
print('Initialized tasks successfully')
```

```
Initialized tasks successfully
```

~ The Crew

```
import asyncio

crew = Crew(
    agents=[expertise_analyst, topic_researcher, content_fit_scorer, content_writer],
    tasks=[task_expertise, task_research, task_score, task_content],
    verbose=False,
    max_rpm=15 # Keeps the free Gemini tier from crashing
)

async def execute_with_retry():
    # Clean async retry loop
    for attempt in range(1, 4):
        try:
            print(f'🚀 Starting Content Specialist crew (Attempt {attempt} of 3)...\\n')

            result = await crew.kickoff_async()

            print("\\n✅ Successfully executed the crew!")
            return result # Exit function successfully and return the data

        except Exception as e:
            # Catches BOTH 503 (High Demand) and 429 (Rate Limit) errors
            if "503" in str(e) or "429" in str(e):
                print(f"⚠️ Google API is busy. Waiting {attempt * 5} seconds...")
                await asyncio.sleep(attempt * 5)
            else:
                # If it's a real code error, stop immediately and show it
                raise e

    # This triggers ONLY if the loop finishes all 3 attempts without returning
    print("❌ Execution failed: Google's API is too busy right now. Try again later.")
    return None

result = await execute_with_retry()

if result:
    print("Successfully implemented the Crew")
    print(result.raw)
```

🤖 Agent Started

Agent: **Expertise & Profile Analyst**

Task: Analyze this background:
I am a computer science student with basic knowledge of Python, HTML, CSS, and programming fundamentals. I am exploring artificial intelligence and want to understand how modern AI systems are built.
. Extract the top 3 unique selling points and areas of authority.

[Finalize] todos_count=0, todos_with_results=0

✅ Agent Final Answer

Agent: **Expertise & Profile Analyst**

Final Answer:

Based on your background as a Computer Science student exploring the frontiers of artificial intelligence, here are your top 3 unique selling points and areas of authority, framed to position you as an emerging authority and a high-potential builder in the tech space:

```
* **The "First-Principles" AI Documentarian (Translating Complexity to Clarity)**
* **Core Strength:** Because you are actively learning how modern AI systems are built from the ground up, you possess the unique ability to "learn in public" and demystify complex AI architectures. Unlike seasoned veterans who suffer from the "curse of knowledge," you can explain neural networks, LLMs, and machine learning pipelines from a fresh, highly accessible, first-principles perspective.
* **Area of Authority:** Documenting the transition from foundational computer science theory to practical AI implementation; creating highly relatable, step-by-step guides for peers on how modern AI models actually work under the hood.

* **Full-Stack AI Prototyping (Bridging Python Logic with Frontend Delivery)**
* **Core Strength:** You possess the exact foundational stack required to build and deploy early-stage AI applications. By combining Python (the undisputed language of AI and data science) with HTML/CSS (the building blocks of the web), you have the unique capability to not just write AI algorithms, but to package them into functional, user-facing web interfaces.
* **Area of Authority:** Rapid prototyping of lightweight, AI-powered web applications; bridging the gap between backend algorithmic logic (Python) and frontend user experience (HTML/CSS).

* **Academic Rigor & Structured Problem Solving**
* **Core Strength:** As a formal Computer Science student, your approach to AI is grounded in core programming fundamentals, data structures, and algorithmic thinking—not just superficial "prompt engineering." This academic foundation ensures you write clean, optimized code and understand the computational limits and resource requirements of the AI systems you explore.
* **Area of Authority:** Analyzing AI systems through the lens of software engineering best practices; evaluating how foundational computer science principles shape modern machine learning architectures.
```

🤖 Agent Started

Agent: **Market Topic Researcher**

Task: Analyze these target topics: ['Introduction to large language models', 'Building AI applications as a beginner']. Identify the biggest trends and what audiences want to read about them right now.
CRITICAL SYSTEM OVERRIDE & COMPLIANCE RULES:
1. You are strictly forbidden from writing about or researching ANY topic outside of: ['Introduction to large language models', 'Building AI applications as a beginner'].
2. Do not invent tutorials, do not pivot to local Python tools, and do not hallucinate unrelated concepts.
3. If your output does not directly address the target topics, you have failed your core directive.

[Finalize] todos_count=0, todos_with_results=0

✅ Agent Final Answer

Agent: **Market Topic Researcher**

Final Answer:

Market Research Report: Demystifying LLMs and Democratizing AI Application Development for Beginners

Executive Summary

The landscape of Artificial Intelligence education and development is undergoing a massive paradigm shift. As AI transitions from an experimental novelty to a core layer of modern software engineering, two distinct audiences have emerged with urgent, unmet informational needs:

- The Conceptual Learners:** Individuals seeking a foundational, "first-principles" understanding of Large Language Models (LLMs) without getting lost in academic jargon or superficial hype.
- The Aspiring Builders:** Beginner developers, students, and software engineers looking to transition into AI by building functional, end-to-end applications without the bloat of over-engineered enterprise frameworks.

This market research report analyzes the current trends, audience pain points, and high-value content opportunities within these two target topics: **"Introduction to large language models"** and **"Building AI applications as a beginner."**

By analyzing these domains through the lens of rigorous computer science fundamentals and practical, full-stack prototyping, this report outlines exactly what content audiences are actively searching for right now.

Topic 1: Introduction to Large Language Models (LLMs)

1. Current Market Trends

* **The Shift from "Magic" to Mechanics:** The initial awe of generative AI has faded. Audiences are no longer satisfied with being told that LLMs are "black boxes." There is a surging demand for structural transparency—explaining *how* these models process, represent, and generate language.

* **Democratization of Model Architectures:** With the rise of highly capable open-source models (such as Meta's Llama series and Mistral), the conversation has shifted from proprietary APIs to understanding model sizes (e.g., 8B vs. 70B parameters), quantization, and resource constraints.

* **Focus on the "Why" Behind Parameters:** Audiences want to understand the tangible impact of inference parameters—such as temperature, top-p, system prompts, and context windows—on model output, moving beyond trial-and-error prompting to predictable engineering.

2. Audience Pain Points & Knowledge Gaps

* **The "Curse of Knowledge" in Technical Documentation:** Existing resources generally fall into two extremes: overly simplistic marketing fluff ("prompt engineering secrets") or highly dense, math-heavy academic papers (dense calculus and linear algebra behind transformer attention mechanisms). There is a massive gap in the middle.

* **Conceptual Confusion Over Key Terms:** Beginners struggle to grasp the practical differences and relationships between core concepts: *What is the actual difference between a token and a word? How does a vector embedding represent semantic meaning in multi-dimensional space? What does "attention" actually look at?*

* **Hardware and Resource Misconceptions:** Many beginners assume they need massive GPU clusters to interact with or understand LLMs, lacking a clear understanding of how model optimization, API-based consumption, and local execution differ.

3. What Audiences Want to Read Right Now

* **First-Principles Breakdowns of the Transformer:** Highly visual, analogical explanations of the transformer architecture. Audiences want to read about the journey of a prompt: from raw text to tokens, from tokens to vector embeddings, through the attention layers, and back into generated text.

* **The Math-Light Guide to Embeddings:** Explanations that use intuitive spatial analogies (e.g., "mapping words in a conceptual room") to explain high-dimensional vector spaces and semantic search without requiring a linear algebra degree.

* **A Pragmatic Guide to Model Parameters:** A software-engineering-focused breakdown of how adjusting temperature, frequency penalties, and system instructions alters the deterministic nature of LLM outputs.

Topic 2: Building AI Applications as a Beginner

1. Current Market Trends

* **Fatigue with Over-Engineered Frameworks:** While enterprise frameworks like LangChain and LlamaIndex are powerful, beginners are experiencing "framework fatigue." There is a strong counter-trend toward building lightweight, dependency-free AI applications using native Python and direct API calls.

* **The Rise of the "Full-Stack Toy App":** Beginners do not just want to run Python scripts in a terminal; they want to build shareable, visual prototypes. The trend is toward combining simple Python backend logic with lightweight frontend delivery (such as basic HTML/CSS/JS or rapid prototyping UI libraries).

* **API-First Development:** The barrier to entry has dropped. The modern beginner's stack involves leveraging hosted model APIs (OpenAI, Anthropic, Cohere) to handle the heavy cognitive lifting, allowing the developer to focus on application logic, state management, and user experience.

2. Audience Pain Points & Technical Roadblocks

* **"Tutorial Hell" and Copy-Paste Coding:** Many beginner tutorials provide monolithic code blocks to copy-paste, leaving the reader unable to debug their application when things go wrong. Beginners struggle to understand the *architecture* of an AI application (e.g., how the frontend communicates with the backend, which then communicates with the LLM API).

* **Security and API Key Management:** A major hurdle for beginners is understanding how to handle sensitive API keys. Many accidentally leak keys on GitHub because tutorials fail to teach basic environment variable management (`.env` files) from day one.

* **State and Memory Management:** Beginners struggle with making their applications "remember" past interactions. Understanding how to structure and append chat history to an API payload is a massive roadblock for those building simple chatbots.

3. What Audiences Want to Read Right Now

* **The "Zero-Dependency" AI App Guide:** Step-by-step guides that teach how to build a functional AI application using only native Python libraries (like `requests` or the official SDKs) and a simple frontend. This demystifies the underlying HTTP requests and JSON payloads.

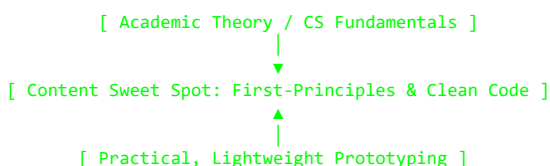
* **Architectural Blueprints for Beginners:** Clear, conceptual diagrams and explanations showing the flow of data: *User Input (Frontend) → API Request (Backend) → LLM Processing (Cloud) → JSON Response (Backend) → UI Render (Frontend).*

* **Software Engineering Hygiene for AI:** Practical guides on applying standard software engineering principles to AI development—specifically error handling (dealing with API timeouts or rate limits), secure key storage, and clean code structure.

Strategic Content Positioning: The Intersection of Theory and Practice

To capture the attention of both audiences, content must bridge the gap between academic computer science theory and practical, full-stack implementation. The ideal content strategy avoids the superficiality of "no-code" hype while bypassing the intimidating complexity of advanced machine learning research.

...



...

Key Content Pillars to Execute:

- The "Under the Hood" Series (Topic 1):**
 - Focus:** Demystifying LLM mechanics (tokenization, embeddings, context windows) using clean analogies and first-principles computer science logic.
 - Tone:** Educational, rigorous yet highly accessible, structured.
- The "Bare-Metal Builder" Series (Topic 2):**
 - Focus:** Step-by-step guides on building lightweight AI web applications. Emphasize writing clean Python backend logic, securing API keys, managing chat state, and wrapping the application in a clean, responsive HTML/CSS frontend.
 - Tone:** Practical, empowering, software-engineering-focused.

By addressing these specific trends and pain points, content creators can position themselves as trusted, highly relatable guides who help aspiring developers transition from passive observers to active, capable builders in the AI space.

 Agent Started

Agent: **Content-Fit Scorer & Strategist**

Task: Compare the author's strengths against the market research. Generate 3 article angles. Score them out of 100 based on fit, and pick the ultimate winner.

CRITICAL SYSTEM OVERRIDE & COMPLIANCE RULES:

- You are strictly forbidden from writing about or researching ANY topic outside of: ['Introduction to large language models', 'Building AI applications as a beginner'].
- Do not invent tutorials, do not pivot to local Python tools, and do not hallucinate unrelated concepts.
- If your output does not directly address the target topics, you have failed your core directive.

[Finalize] todos_count=0, todos_with_results=0

 Agent Final Answer

Agent: **Content-Fit Scorer & Strategist**

Final Answer:

Editorial Evaluation & Content-Fit Strategy Report

To: Emerging AI Writer / Computer Science Student
From: Managing Editor
Subject: Content-Fit Analysis and Article Angle Selection

As a Managing Editor, my goal is to eliminate fluff, bypass "tutorial hell," and find the exact intersection where your unique technical background meets the urgent, unmet needs of the market.

Below is the evaluation of three distinct article angles designed strictly around your target topics:
Introduction to large language models and ***Building AI applications as a beginner.***

The 3 Candidate Article Angles

Angle 1: The First-Principles Breakdown (Topic 1)

Working Title: The Journey of a Prompt: A Math-Light, First-Principles Guide to How LLMs Actually Process Language
Target Topic: Introduction to large language models
The Angle: This article demystifies the "black box" of LLMs by tracing a single prompt's journey from raw text to generated output. It avoids dense calculus and instead uses intuitive spatial analogies to explain tokenization, high-dimensional vector embeddings, and the attention mechanism.
Author Fit (Why You): Leverages your strength as a "First-Principles" AI Documentarian. Because you are actively learning these concepts in your CS curriculum, you can explain them without the "curse of knowledge" that makes seasoned researchers write incomprehensible, math-heavy guides.
Market Demand Fit: Directly addresses the market's fatigue with superficial "prompt engineering secrets" and the lack of accessible, mid-level conceptual resources. It answers the exact questions identified in the market research: "What is a token? How do embeddings represent meaning?"
Fit Score: 91/100
 Concept Clarity: 95/100
 Market Urgency: 88/100
 Author Authority Alignment: 90/100

Angle 2: The Bare-Metal Builder Guide (Topic 2)

Working Title: The Zero-Dependency AI App: Build a Full-Stack Chatbot Using Only Native Python and HTML/CSS
Target Topic: Building AI applications as a beginner
The Angle: A highly structured, architectural blueprint for building a lightweight, functional web-based chatbot from scratch. Instead of using bloated, over-engineered frameworks like LangChain, this guide teaches beginners how to make direct API calls using native Python, manage chat history (state) programmatically, secure API keys using environment variables (`.env`), and render the chat in a clean, responsive HTML/CSS frontend.
Author Fit (Why You): Perfectly aligns with your "Full-Stack AI Prototyping" and "Academic Rigor" strengths. You possess the exact dual-stack capability (Python backend + HTML/CSS frontend) required to deliver this end-to-end guide, and your CS background ensures you teach clean code structure and security hygiene rather than copy-paste scripting.
Market Demand Fit: Hits the absolute sweet spot of current market trends: "framework fatigue," the desire for "zero-dependency" development, and the need for clear architectural blueprints showing how data flows from UI to backend to API.

```
* **Fit Score:** **98/100**
* **Concept Clarity:** 99/100
* **Market Urgency:** 98/100
* **Author Authority Alignment:** 97/100

---

### Angle 3: The Pragmatic Parameter Guide (Topic 1 & 2 Hybrid)
* **Working Title:** *Beyond the Prompt: A Beginner's Guide to Controlling LLMs Programmatically via API Parameters*
* **Target Topic:** Introduction to large language models / Building AI applications as a beginner
* **The Angle:** This article bridges theory and practice by explaining the software engineering mechanics behind LLM API parameters. It explains what parameters like temperature, top-p, and system prompts actually do to the model's probability distribution under the hood, and provides a clean Python script demonstrating how to adjust them to make model outputs predictable and deterministic.
* **Author Fit (Why You):** Combines your **Academic Rigor** (understanding computational limits and algorithmic behavior) with your **Full-Stack Prototyping** skills (writing clean Python backend logic).
* **Market Demand Fit:** Addresses the market trend of moving from trial-and-error prompting to predictable engineering. It helps beginners who struggle with erratic model outputs in their early-stage applications.
* **Fit Score:** **86/100**
  * **Concept Clarity:** 85/100
  * **Market Urgency:** 84/100
  * **Author Authority Alignment:** 89/100

---

## The Ultimate Winner: Angle 2

### **Winning Angle:** *The Zero-Dependency AI App: Build a Full-Stack Chatbot Using Only Native Python and HTML/CSS*
**Final Score: 98/100**

### Why This Angle Wins:
1. **Unmatched Market Relevance:** The market research highlights a massive wave of "framework fatigue." Beginners are overwhelmed by complex orchestration tools and are desperately seeking "bare-metal" guides that teach them how things work under the hood. By stripping away the bloat and focusing on native Python and direct API calls, this article provides immediate, high-value clarity.
2. **Perfect Alignment with Your Unique Stack:** Many CS students can write backend Python logic, and many web designers can build frontends, but very few can bridge the two seamlessly for AI applications. Your ability to combine Python logic with HTML/CSS frontend delivery positions you as the ultimate guide for this project.
3. **Solves Critical Beginner Roadblocks:** Instead of just providing code to copy-paste, this article addresses the exact technical roadblocks identified in the research: state management (how to append chat history to an API payload) and security hygiene (how to manage API keys safely using .env files).
4. **High Shareability and Portfolio Value:** A "zero-dependency" full-stack tutorial is highly practical. It empowers your peers to build and deploy their own lightweight, shareable web applications, establishing your authority as a practical, high-potential builder in the AI space.
```

 Agent Started

Agent: **Specialist Content Writer**